

Обслуживание LLaMA 3-70B на TPU

Часть 8 из [Как масштабировать свою модель](#) ([Часть 7: Выводы](#) | [Часть 9: Профилирование](#))

Давайте подробно рассмотрим, как мы будем обслуживать модели LLaMA 3-70B на TPU v5e. Насколько дорого обходится обслуживание различных моделей на пределе возможностей? Насколько велики их кэши KV? Какой размер пакетов следует использовать? Как распределяются параметры и активации при логическом выводе? Давайте прикинем приблизительные значения задержки и пропускной способности в производственной среде.

АВТОРЫ
[Джейкоб Остин](#)
[Шолто Дуглас](#)
[Рой Фростиг](#)
[Ансельм Левская](#)
[Чарли Чен](#)
[Шарад Викрам](#)

[Федерико Леброн](#)
[Питер Чой](#)
[Винай Рамасеш](#)
[Альберт Уэбсон](#)
[Райнер Поуп](#)

МЕСТО РАБОТЫ
Google DeepMind

ОПУБЛИКОВАНО
4 февраля 2025 года

В этом разделе мы рассмотрим, что нужно для обслуживания LLaMA-3 и насколько эффективно это можно делать. Как и в предыдущем «прикладном» разделе, попробуйте найти ответы самостоятельно, с помощью ручки и бумаги, прежде чем искать их в интернете!

Что представляет собой сервисная история LLaMA?

Давайте вспомним, как выглядит LLaMA 3-70B (см. [Раздел 6](#)):

гиперпараметр	значение
n_{layers} (L)	80
d_{model} (D)	8,192
d_{ff} (F)	28,672
n_{heads} (N)	64
$n_{\text{kv heads}}$ (K)	8
d_{qkv} (H)	128
$n_{\text{embeddings}}$ (V)	128,256

Давайте начнём с простого вопроса: **на каком оборудовании нам следует работать?** Ответ прост: на том, которое дешевле всего в пересчёте на гигафлопс на доллар.¹ По этой причине мы обычно предпочитаем использовать TPU v5e, наш текущий специализированный чип для логического вывода (стоимость указана по [тарифам Google Cloud](#) на февраль 2025 года):

Тип TPU	bfloat16 FLOPs/c	Google Cloud в долларах США / час	Провалы / \$
H100	9,9e14	\$10.8	3.3e17
v5p	4,59e14	\$4.2	3,9e17
v5e	1,97e14	\$1.2	5,8e17

Каждый TPU v5e оснащен 16 ГБ памяти HBM, поэтому нам придется довольно агрессивно сегментировать нашу модель. Для начала давайте подумаем о некоторых базовых показателях, которые могут иметь для нас значение:

Вопрос: Каков размер KV-кэша LLaMA 3-70B на токен? Можно предположить, что мы храним их в виде `int8`. Это определяет, насколько большим может быть размер пакета в данной топологии.

► [Нажмите сюда, когда все обдумаете!](#)

Вопрос: Допустим, мы хотим использовать L3 70B с размером пакета 32 и длиной последовательности 8192, где все (параметры и ключевые слова) представлены в виде `int8`. Сколько памяти это займет? Какой наименьший фрагмент мы могли бы использовать?

► [Ответ](#)

Вопрос: При таком размере пакета и квантовании на TPU v5e 4x2 примерно какую задержку мы можем ожидать на каждом этапе декодирования? Какая будет пропускная способность (токены в секунду на чип)? А как насчет 4x4? *Предположим, что мы выполняем операции над числами с плавающей запятой в формате bfloat16 и все данные полностью сегментированы.*

► Ответ

Размышления о пропускной способности

Давайте немного поразмышляем исключительно о пропускной способности. Когда мы оптимизируем ее, мы хотим, чтобы она зависела от вычислительных ресурсов, то есть чтобы мы максимально использовали возможности TPU MXU. Как правило, это означает, что нам нужен как можно больший размер пакета, чтобы выполнять как можно больше работы.

Вопрос: При использовании весов и активаций bfloat16 на TPU v5e, насколько большими должны быть размеры наших пакетов, чтобы мы не зависели от вычислительных ресурсов при выполнении матричных умножений? Что, если мы используем веса int8, но выполняем операции над числами с плавающей запятой в формате bfloat16? А что, если мы используем веса int8 и операции над числами с плавающей запятой в формате int8?

► Ответ

Вопрос: Какую минимальную топологию TPU v5e мы могли бы использовать для LLaMA 3-70B, используя bfloat16, int8 и int4 (как ключевые слова, так и параметры) с контекстом 8k? *Для этого можно считать, что кэш ключевых слов пренебрежимо мал.*

► Ответ

Вопрос: Предположим, что мы используем максимальный размер пакета, который подходит для этих топологий. Какую задержку можно ожидать на каждом этапе генерации?

► Ответ

Вывод: мы всегда можем снизить верхнюю границу задержки декодирования, спросив, сколько времени требуется для загрузки всех параметров модели из HBM в MXU. Если наши кэши KV небольшие, то можно представить, что каждый слой просто загружает веса по частям, а затем отбрасывает их. Если мы не используем большие размеры пакетов или множество межпроцессорных соединений, то это часто является разумной границей (в пределах 1,5 раза). Когда размер пакета данных увеличивается, нам также нужно смоделировать загрузку KV-кэша, поскольку это основной параметр.

Аналогичным образом в режиме, ограниченном количеством операций с плавающей запятой (например, при обучении или логическом выводе с большими пакетами данных), мы можем использовать $\text{Total FLOPs} / (N \cdot C) = 2 \cdot \text{param count} \cdot B / (N \cdot C)$ Нижняя граница, предполагающая отсутствие связи.

Вопрос: Какую пропускную способность на чип мы получим для каждого из них (в пересчете на количество запросов на чип)? *Можно предположить, что средняя длина декодирования составляет 512 токенов.*

► Ответ

Вопрос: Как изменится наша пиковая пропускная способность, если мы удвоим топологию для каждого из приведенных выше примеров?

► Ответ

Вопрос: Теперь давайте разберемся с шардированием. Допустим, мы хотим использовать bfloat16 на TPU v5e 4x8. Какое шардирование мы будем использовать для нашей модели на TPU v5e 4x8 во время генерации? Можно ли избежать проблем с коммуникацией?

► Ответ

Tip: the maximum amount of useful model parallelism depends on d_{ff} and the number of axes over which you're sharding your model. The maximum value usually ranges between 8 and 32 depending on the model size. You can scale beyond this limit to improve latency at some throughput cost.

What about prefill?

We've mostly ignored prefill here because it's much simpler. Let's put a couple of concepts together and think about the end-to-end picture.

Question: Assume we achieve a 40% FLOPs utilization during prefill. How long will a prefill of length 8192 take on 16 TPU v5e chips?

► Answer

Question: Assume we have a median prefill length of 8192 tokens and a median decode length of 4096 tokens. Say we have a generate batch size of 32. On average how many sequences finish decoding per step? On average how many tokens are evicted from our KV cache each step?

► Answer

Question: Assume we do disaggregated serving with a median prefill length of 8192 and a median decode length of 512. Assume the prefill and generate latencies calculated above in bfloat16. What ratio of prefill:generate servers will you need to keep both fully saturated.

► Answer

Visualizing the Latency Throughput Tradeoff

Sticking with LLaMA 70B for a second, let's actually look at the latency and throughput for different batch sizes during generation. As we showed in the previous section for PaLM models, this gives us a Pareto frontier for throughput/latency. Let's assume 16-way tensor parallelism since that's a reasonable bound on what we can use while staying compute-bound in the MLP blocks. We'll use a TPU v5e 4x4 topology here. **The slider controls the sequence length so you can see the effect of larger KV caches.**

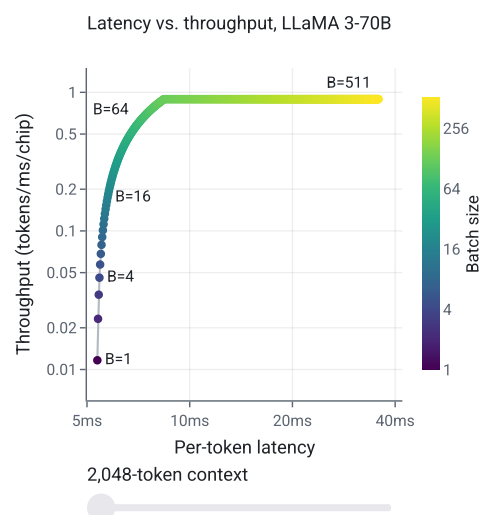


Figure: throughput per chip against per-token latency for LLaMA 3-70B (int8 weights, 16-way tensor parallelism) on a TPU v5e 4x4, sweeping batch size along each curve. Drag the slider to change the context length; each curve stops at the largest batch whose KV cache fits in HBM.

- **See how dramatic the tradeoff is between cost and latency.** At the cost of doubling per-token latency, we can achieve a roughly 100x reduction in per-token cost. Also, our latency can range anywhere from 5.5ms with low batch size to 20 ms with very large batches.
- Note how at 2k context the throughput effectively plateaus at around 1 token / ms / chip when it hits the BS 120 roofline (120 here because we do int8 weights but bf16 FLOPs). As the sequence length increases, however, we can no longer fit this batch size in memory, so we never hit the point of full saturation.
- Note how much higher the latency is at large batch sizes for the same throughput, since KV loading becomes dominant (instead of parameter loading).

We can understand this better by breaking down the sources of cost and latency into param loading time, KV loading time, and FLOPs time. The shaded region is where we expect to be compute-bound in our MLP blocks.

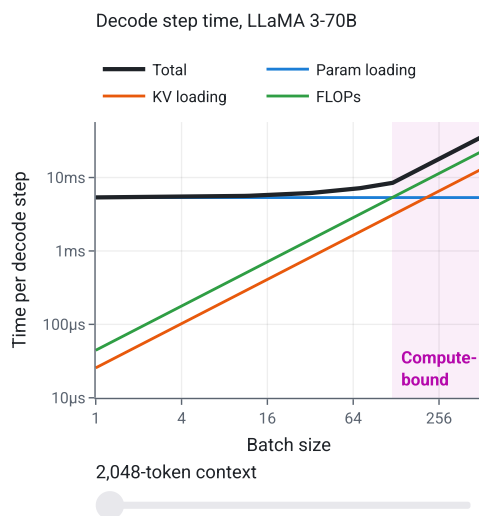


Figure: time per decode step for LLaMA 3-70B on a TPU v5e 4x4 as a function of batch size, split into parameter loading, KV cache loading, and FLOPs. The total is the KV loading time plus whichever of parameter loading or FLOPs is larger. Drag the slider to change the context length.

This tells quite a story. You can see that initially, parameter loading represents the vast majority of the latency, until the batch size becomes large enough that FLOPs and KV loading become more significant. Notably, at all sequence lengths greater than 2048, we spend more time on KV cache loading than we do on FLOPs! **So while we can improve our hardware utilization by increasing batch size, at long context lengths KV loading always dominates the total step time.**

Takeaway: for LLaMA 3-70B, we are strongly KV cache memory bandwidth-bound (and HBM-bound) in almost all of these configurations, highlighting just how important reducing KV cache size is for generation throughput. Also note just how dramatic the latency/throughput tradeoff remains here.

► The code for this is quite simple.

Worked Problems

Here are a few worked problems. Some of these repeat things that are worked above, but might be pedagogically useful.

Question 1: How many FLOPs does each forward pass for LLaMA 3-405B use per-token? Assuming we're FLOPs bound, what is a lower bound on a single forward pass on N chips on TPU v5e? What if we're comms bound? *Ignore the fact that the model does not fit on a single chip.*

Question 2: Assume we want to serve LLaMA 3-8B with BS240 using int8 weights and int8 KV caches. How many bytes are used by (a) model parameters (b) KV caches and (c) peak working activations (roughly)? What's the smallest topology we can run this on?

Question 3: How would you serve LLaMA 3-405B on TPU v5e? Assume int8 weights and bfloat16 FLOPs. Let's say we have a firm limit of 15ms / token, what's the highest throughput configuration we could achieve? What is the theoretical minimum step time?

Question 4: The best way to learn about LLMs is to implement one from scratch. Building the full training pipeline is annoying and expensive, but turning trained weights you can [download from HuggingFace](#) into a working inference implementation is extremely instructive. Roughly speaking, you should try to do the following:

1. Download LLaMA 3 8B weights and load them in Colab. Visualize the weights (I like [TreeScope](#) for this) and see if you can identify each tensor. Count the # of parameters. Does it match what you expect? How many are in the MLP vs. attention?
2. Implement the full forward pass of the model. You don't need to worry about prefill/decode or anything fancy. Just get to the point where you can feed in a sequence and get plausible next-token probabilities out. You should be able to feed a prompt in, get probabilities out, pick the highest probability one, and repeat. This is slow but functional sampling. Try to only use the LLaMA-3 paper as a reference, but the [jax-ml/jax-llm-examples/llama3](#) repo can be a good reference for details (be careful of positional embeddings and causal masking). Your goal is to get coherent tokens out of the model. *You can run this all on a single TPU if you can get more than 16GiB of HBM.*
3. Now it's time to make this faster. Implement KV caching, where you can save the key/value activations from a forward pass and attend to them in a subsequent one. This will make your sampling loop much faster.
4. Implement separate prefill and decode servers. You can do these on different sets of chips. Prefill handles just a single prompt at once, then sends it to a batched decode server that handles batches of tokens.
5. Implement Flash Attention in Pallas. This will make attention much more efficient. *I think it's good to try and do this without a reference.*

That's all for Part 8! For Part 9, with a deep dive into XLA and TPU profiling, click [here](#).

Footnotes

1. Это не всегда так, иногда важнее пропускная способность HBM или ICI, а не количество операций с плавающей запятой, но это хорошая эвристика. [↗]

Miscellaneous

*Work done at Google DeepMind, now at MatX.

Citation

For attribution in academic contexts, please cite this work as:

Austin et al., "How to Scale Your Model", Google DeepMind, online, 2025.

or as a BibTeX entry:

```
@article{scaling-book,
  title = {How to Scale Your Model},
  author = {Austin, Jacob and Douglas, Sholto and Frostig, Roy and Levkaya, Anselm and Chen, Charlie and Vikram, Sharad
and Lebron, Federico and Choy, Peter and Ramasesh, Vinay and Webson, Albert and Pope, Reiner},
  publisher = {Google DeepMind},
  howpublished = {Online},
  note = {Retrieved from https://jax-ml.github.io/scaling-book/},
  year = {2025}
}
```